

Evidencia de pruebas

Billetera Android Kotlin con bundles React Native

JHON ALEXANDER SUAREZ OSSA

Prueba técnica · Desarrollador React Native

Septiembre de 2026

Contenido

Cómo está probado el proyecto.....	3
El entorno exacto de las corridas	3
Los tres niveles de prueba y sus cifras	4
Por qué las capturas son del APK de depuración.....	4
Cómo repetir las corridas	4
Las once pruebas mínimas del enunciado	6
Pruebas unitarias de Kotlin	8
Qué cubre cada clase	9
BancoMockApiTest, 17 pruebas.....	9
AesGcmTest, 5 pruebas.....	9
BridgeEnvelopeTest, 6 pruebas.....	10
PasswordHasherTest, 6 pruebas	10
SesionManagerTest, 6 pruebas	11
La única prueba omitida.....	11
Pruebas de Jest.....	12
Qué cubre cada archivo	13
El canal cifrado: crypto.test.ts, 8 pruebas.....	13
La transferencia: useTransferencia.test.ts, 5 pruebas	14
Las tres pantallas: 7 pruebas	14
Formato, validaciones y agrupación: 23 pruebas	14
Pruebas instrumentadas en el emulador	15
Cómo se espera a React Native sin pruebas frágiles.....	16
El recorrido completo en capturas.....	17
La compilación de publicación	28
R8 renombró y borró.....	28
El APK va firmado	28
Los permisos de cada variante	29
El recorrido completo repetido sobre el APK ofuscado	29

Cómo está probado el proyecto

Este documento reúne la evidencia de las pruebas de la billetera: qué se probó, con qué comando, con qué resultado y qué quedó guardado como respaldo. Todo salió de correr el proyecto en mi equipo. No hay cifras estimadas. Cada número que aparece aquí viene de un archivo de salida que está en la carpeta docs/evidencia/ del repositorio.

Uso tres términos a lo largo del texto. Una **prueba unitaria** revisa una pieza suelta del código, sin pantalla y sin dispositivo: es como probar un motor en el banco, no el carro andando. Una **prueba de componente** monta una pantalla de React Native en memoria y la opera como lo haría una persona, pero dentro de Node, el entorno de JavaScript del computador. Una **prueba instrumentada** se instala en un Android y corre allá adentro, dentro del proceso de la aplicación.

El entorno exacto de las corridas

Las pruebas corrieron sobre una sola máquina y un solo dispositivo virtual. Estas son las versiones, tomadas de los archivos de configuración del proyecto y de la consola.

Pieza	Versión
Sistema operativo	Windows 11
JDK	17
Gradle	8.14.3, el wrapper que trae el proyecto
Kotlin	2.1.20
Android SDK	compileSdk 36, targetSdk 36, minSdk 26, build tools 36.0.0
Node.js	24.18
React Native	0.81.6 con React 19.1.4
Motor de JavaScript	Hermes, con los bundles precompilados
Pruebas en la JVM	JUnit 4.13.2 y org.json 20260814
Pruebas instrumentadas	androidx.test 1.7.0, ext.junit 1.3.0, Espresso 3.7.0
Pruebas de JavaScript	Jest 29 y Testing Library React Native 14.0.1
Emulador	anden_demo, Android 16 (API 36), google_api x86_64, visto por adb como emulator-5554

El emulador es un Android virtual que corre dentro del computador. anden_demo es el nombre que le puse al perfil y emulator-5554 es la dirección con la que lo ve adb, la herramienta de línea de comandos de Android.

Los archivos de docs/evidencia/ son las salidas tal cual, sin retocar. Lo único que les hice fue pasarlas a UTF-8, porque la consola de Windows escribe en cp1252 y las tildes quedaban rotas.

Los tres niveles de prueba y sus cifras

El proyecto tiene tres niveles de prueba automática. Cada uno se lanza con un comando y deja su propia salida.

Nivel	Comando	Dónde corre	Cifras reales
Unitarias de Kotlin	gradlew testDebugUnitTest	En la JVM del computador, sin dispositivo	40 pruebas en 5 clases. 0 fallos, 0 errores, 1 omitida a propósito
Jest, JavaScript y TypeScript	npm test	En Node, sin dispositivo	43 pruebas en 8 archivos. 0 fallos
Instrumentadas de Android	gradlew connectedDebugAndroidTest	Dentro del emulador anden_demo	13 pruebas en 8 clases. 0 fallos, 0 omitidas

Son 96 pruebas automáticas y todas pasan. El único caso omitido es un generador de hashes que dejé apagado a propósito, y lo explico en el capítulo de Kotlin. A eso se suma el recorrido manual sobre el emulador, que es de donde salen las capturas, y ese mismo recorrido repetido sobre el APK de publicación.

Por qué las capturas son del APK de depuración

Las capturas de pantalla están tomadas sobre el APK **debug**. En la variante de publicación la ventana lleva FLAG_SECURE, la bandera de Android que impide capturar la pantalla y que oculta la aplicación del conmutador de tareas. Con esa bandera puesta, adb shell screencap devuelve la ventana en negro, que es exactamente para lo que está ahí.

El flujo de la variante de publicación lo verifiqué igual, pantalla por pantalla, leyendo el árbol de la interfaz con uiautomator, una herramienta de Android que vuelca en XML todos los textos que hay en pantalla. Esas lecturas están en los archivos smoke-release-*.txt y las resumo en el capítulo 7.

Cómo repetir las corridas

Con el repositorio clonado y el emulador encendido, esta es la secuencia completa.

```
npm install
npm run bundles
npm test

cd android
gradlew testDebugUnitTest
gradlew connectedDebugAndroidTest
```

Las pruebas instrumentadas necesitan los cuatro bundles compilados en `android/app/src/main/assets/bundles/`, así que `npm run bundles` va antes. Un bundle es el JavaScript de una pantalla empaquetado en un solo archivo. Aquí van precompilados a Hermes: los cuatro pesan entre 1,62 y 1,64 MB y empiezan por los ocho bytes mágicos `c6 1f bc 03 c1 03 19 1f`, que son la firma de ese formato. Quedó comprobado en `docs/evidencia/bundles-verification.txt`.

Las once pruebas mínimas del enunciado

El enunciado pide once pruebas mínimas. La tabla siguiente cruza cada una con la prueba automática que la cubre y con la figura del capítulo 6 que la muestra. Los nombres de clase y de método son los reales del repositorio. Los de Kotlin usan frases entre acentos graves, que es una forma válida de nombrar métodos de prueba en ese lenguaje y se lee mejor en los informes.

Prueba pedida	Prueba automática que la cubre	Evidencia visual
Carga correcta del bundle de Login	LoginActivityCargaBundleTest.montaElBundleDeLogin (instrumentada): espera el contexto de React y que el root view tenga hijos	login.png
Login exitoso y fallido	BancoMockApiTest.login con la clave correcta devuelve el usuario y BancoMockApiTest.celular desconocido y clave equivocada dan el mismo error (JVM). LoginBundle.test.tsx, “muestra el mensaje del código que rechaza el nativo” (Jest)	home.png y login-error.png
Creación de sesión	SesionManagerTest.crear deja la sesión guardada y vigente (JVM). SesionPersistenciaTest.otraInstancia_leeLaSesionQueQuedoGuardada (instrumentada, sobre el almacenamiento cifrado real)	home.png
Carga del bundle Home	HomeActivityCargaBundleTest.conSesionValidaMontaHomeYAceptaElHandshake (instrumentada). HomeBundle.test.tsx, “avisa que ya escucha y pinta lo que llega en LOAD_HOME” (Jest)	home.png
Apertura del bundle Transferencia	NavegacionDesdeHomeTest.openTransferAbreLaPantallaDeTransferencia (instrumentada)	transferencia-paso1.png
Apertura del bundle Movimientos	NavegacionDesdeHomeTest.openMovementsAbreLaPantallaDeMovimientos (instrumentada)	movimientos.png
Cifrado y descifrado de información	AesGcmTest.cifra y descifra el mismo contenido y BridgeEnvelopeTest.descifra el envelope que generó el lado de JS (JVM). crypto.test.ts, “descifra el envelope del archivo” (Jest, mismo vector). SecureStoreKeystoreTest.elValorCrudoNoEsElTextoPlano (instrumentada)	No tiene pantalla propia, va explicado abajo
Sesión expirada	SesionManagerTest.cuando pasa el ttl la sesión deja de valer y se borra (JVM, con reloj inyectado). HomeSinSesionRedirigeTest.sinSesionNoAbreHome (instrumentada)	sesion-expirada.png
Logout	LogoutLimpiaSesionTest.logoutBorraLaSesionYVuelveALogin (instrumentada). SesionManagerTest.cerrar borra lo que había guardado	login.png, que es a donde

	(JVM). HomeBundle.test.tsx, “cerrar sesión se lo deja al contenedor” (Jest)	vuelve
Transferencia exitosa	BancoMockApiTest.una transferencia exitosa deja debito en el origen y credito en el destino (JVM). useTransferencia.test.ts, “va del celular al resultado y le avisa al contenedor” (Jest)	transferencia-confirmacion.png y transferencia-listo.png
Transferencia fallida por saldo insuficiente	BancoMockApiTest.sin saldo suficiente queda el movimiento rechazado y no se mueve la plata (JVM). useTransferencia.test.ts, “con saldo insuficiente cierra la hoja y deja corregir el monto” (Jest)	saldo-insuficiente.png

El cifrado es la única de las once que no se ve en una captura, y por una razón de fondo: el canal va cifrado de punta a punta, así que en pantalla se ve el resultado y nunca el sobre. La prueba de que los dos lados hablan el mismo formato es el archivo `test-vectors/aes-gcm.json`. Kotlin descifra el sobre que produjo JavaScript y, con la misma llave y el mismo vector de inicialización, reproduce byte a byte el mismo campo `data`. Jest hace lo mismo contra ese archivo. Del lado del almacenamiento, `SecureStoreKeystoreTest.elValorCrudoNoEsElTextoPlano` guarda un celular, lo lee crudo de las `SharedPreferences` y comprueba que ahí no está el texto plano.

Pruebas unitarias de Kotlin

Estas pruebas corren en la máquina virtual de Java del computador, sin emulador. Son las más rápidas: los 40 casos tardan menos de lo que tarda el proyecto en compilarse. Viven en `android/app/src/test/java/com/asuarez/billetera/` y usan JUnit 4.

Como no hay un Android detrás, el módulo declara `unitTests.returnDefaultValues = true`, que hace que las clases del sistema devuelvan valores por defecto en vez de lanzar una excepción. Por eso el módulo también trae `org.json` como dependencia de prueba: el `JSONObject` de Android es un esqueleto vacío en esta JVM, y las pruebas de sesión y de sobre necesitaban la implementación de verdad.

Dos piezas se sustituyen para que las pruebas no dependan del dispositivo. El almacenamiento se reemplaza por `StoreEnMemoria`, que implementa la misma interfaz `KeyValueStore` pero guarda en un mapa. Y el reloj se inyecta: `SesionManager` y `BancoMockApi` reciben una función que devuelve la hora, así que la expiración se prueba moviendo un número y no esperando dos minutos frente a la pantalla.

```
cd android
gradlew testDebugUnitTest
```

```
> Task :app:compileDebugUnitTestKotlin
> Task :app:testDebugUnitTest
```

```
BUILD SUCCESSFUL in 9s
36 actionable tasks: 4 executed, 32 up-to-date
```

40 pruebas, 0 fallos, 0 errores, 1 omitida. El conteo por clase salió de los informes XML que Gradle deja en `app/build/test-results/testDebugUnitTest/`.

```
BancoMockApiTest    tests=17 failures=0 errors=0 skipped=0
AesGcmTest          tests=5  failures=0 errors=0 skipped=0
BridgeEnvelopeTest  tests=6  failures=0 errors=0 skipped=0
PasswordHasherTest tests=6  failures=0 errors=0 skipped=1
SesionManagerTest   tests=6  failures=0 errors=0 skipped=0
TOTAL               tests=40 failures=0 errors=0 skipped=1
```

Clase	Pruebas	Qué comprueba
BancoMockApiTest	17	Las reglas de negocio: login, registro, transferencia y movimientos
AesGcmTest	5	El cifrado AES-256-GCM: ida y vuelta, tamaños, detección de manipulación y no reuso del IV
BridgeEnvelopeTest	6	El sobre que viaja por el puente, el vector compartido con JavaScript y la rotación de la llave
PasswordHasherTest	6, una	El hash PBKDF2 de las claves: formato, verificación y rechazo de

	omitida	valores inválidos
SesionManagerTest	6	Creación, vigencia, expiración por tiempo, cierre y qué queda escrito en disco

Qué cubre cada clase

BancoMockApiTest, 17 pruebas

Es la clase más grande porque ahí están las reglas de negocio. BancoMockApi es el backend simulado del proyecto: no hay servidor, los datos viven en el almacenamiento cifrado del dispositivo y las reglas son las que pide el enunciado.

- Login: la clave correcta devuelve el usuario con su saldo. El celular desconocido y la clave equivocada devuelven el mismo código, CREDENCIALES_INVALIDAS, para no regalar qué números tienen cuenta.
- Una cuenta que no está en estado ACTIVO no entra: responde USUARIO_INACTIVO.
- Registro: abre la cuenta con el abono de bienvenida y la deja en estado ACTIVO. Rechaza el celular que ya tiene cuenta, la clave de menos de seis caracteres, el celular mal formado y los campos vacíos.
- Transferencia exitosa: deja un débito en el origen, un crédito en el destino, la misma fecha en los dos movimientos y el nuevo saldo calculado del lado nativo.
- Transferencia sin saldo: no mueve la plata y deja el movimiento en estado RECHAZADA, que es lo que después se ve en la lista.
- Los demás rechazos: transferirse a sí mismo, destino que no existe, destino inactivo y monto en cero o negativo.
- Movimientos: solo los del usuario autenticado y el más reciente primero.
- Claves: ni la respuesta ni lo persistido llevan la clave en texto plano, y el hash guardado empieza por pbkdf2\$210000\$.
- Persistencia: el estado sobrevive a una instancia nueva sobre el mismo almacenamiento, y un valor corrupto hace que se vuelva a los datos semilla en vez de reventar.

AesGcmTest, 5 pruebas

AES-256-GCM es el algoritmo con el que se cifra todo lo que cruza entre Kotlin y los bundles. GCM, además de cifrar, firma: si alguien cambia un solo byte, el descifrado falla en vez de devolver basura.

- Ida y vuelta: el texto sale igual al que entró, con un IV de 12 bytes y un texto cifrado que pesa 16 bytes más que el plano. Esos 16 bytes son el tag de autenticación.
- Un texto cifrado manipulado lanza AEADBadTagException.
- Dos mensajes iguales salen distintos, porque el IV cambia en cada cifrado.

- Veinte cifrados seguidos no repiten IV. Reusar el IV con la misma llave es el error clásico de GCM.
- `cifrarConIv` descifra lo que cifró, que es lo que permite comparar contra el vector compartido con JavaScript.

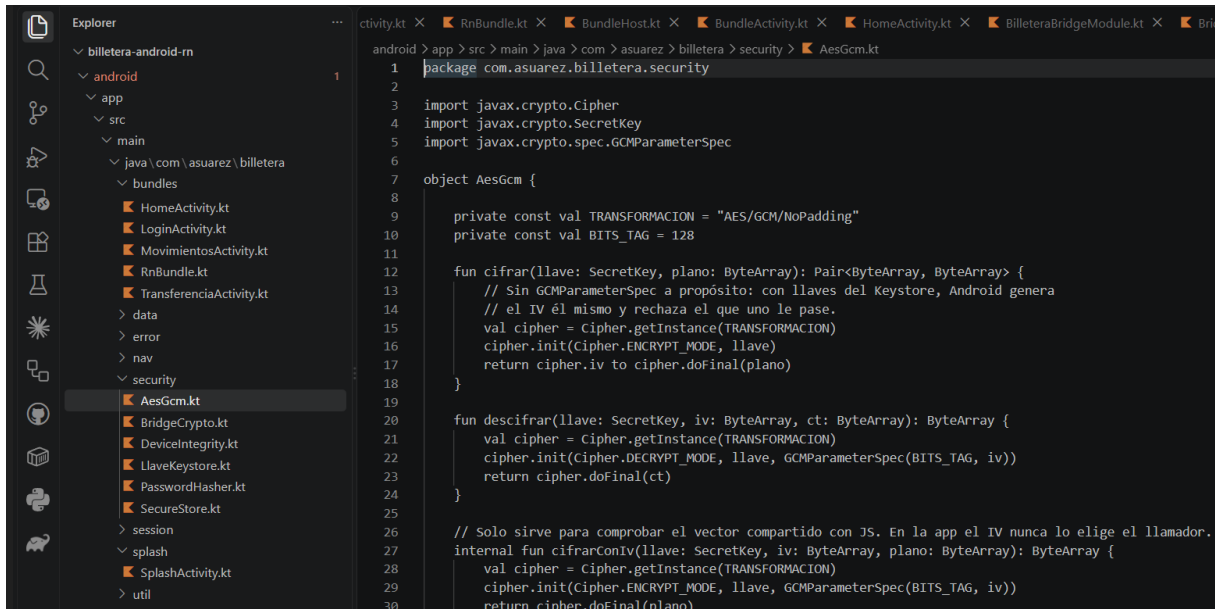


Figura 1. La clase `AesGcm`, que es lo que ejercita `AesGcmTest`: IV nuevo en cada cifrado y tag de 128 bits al final del texto cifrado.

BridgeEnvelopeTest, 6 pruebas

BridgeCrypto es la pieza que arma el sobre que viaja por el puente: un JSON con dos campos, `iv` y `data`, los dos en base64.

- Descifra el sobre que generó el lado de JavaScript, guardado en `test-vectors/aes-gcm.json`.
- Con el mismo IV y la misma llave produce exactamente el mismo data que trae el archivo. Es la prueba dura de que los dos lados implementan lo mismo.
- El sobre de salida trae IV de 12 bytes y data no vacío, y se descifra a sí mismo devolviendo el texto original.
- Un data alterado y un sobre sin data levantan `PayloadInvalido`, la excepción propia que no arrastra el contenido del mensaje.
- `rotar()` cambia la llave del canal, y la nueva sigue midiendo 32 bytes.

La llave del canal no está escrita en el código. Se genera con `SecureRandom` al arrancar, vive solo en memoria, se le entrega al bundle en las props iniciales y se rota al volver a Login.

PasswordHasherTest, 6 pruebas

PBKDF2 convierte una clave en un hash y lo hace a propósito lento, repitiendo la operación 210.000 veces. Así, si alguien se roba el archivo, probar claves una por una le cuesta caro. El salt es

un valor aleatorio que se guarda junto al hash para que dos personas con la misma clave no terminen con el mismo hash.

- La clave con la que se generó verifica. Otra clave, la misma con mayúsculas distintas y la cadena vacía no verifican.
- El formato guardado tiene cuatro partes separadas por el signo de pesos: pbkdf2, las 210.000 iteraciones, un salt de 16 bytes y un hash de 32.
- La misma clave da hashes distintos en cada llamada, porque el salt es nuevo.
- Un valor guardado que no tiene forma de hash devuelve falso, en vez de lanzar una excepción.

SesionManagerTest, 6 pruebas

- crear deja la sesión guardada y vigente, con la expiración puesta a dos minutos del reloj inyectado.
- Dos sesiones seguidas no comparten sessionId, y el identificador mide 36 caracteres, que es el largo de un UUID.
- Cuando pasa el tiempo de vida, actual() devuelve nulo, esValida() da falso y el almacenamiento queda vacío. La sesión vencida no se queda ahí guardada.
- cerrar borra lo que había guardado.
- El JSON escrito trae sessionId, name, phone y expiresAt, y no trae ni clave, ni claveHash, ni la cadena pbkdf2 por ningún lado.
- Un valor corrupto en el almacenamiento se descarta en vez de reventar la aplicación.

La única prueba omitida

PasswordHasherTest.genera los hashes de los usuarios semilla es la única prueba que aparece como omitida, y lo es a propósito. Es el generador con el que saqué los hashes PBKDF2 que están fijos en DatosSemilla. Arranca con `assumeTrue(System.getProperty("semilla") == "true")`, así que solo corre si se lanza con `-Dsemilla=true`. No tiene por qué ejecutarse en cada compilación, y por eso el conteo muestra `skipped=1`.

Las salidas completas están en `docs/evidencia/pruebas-kotlin.txt` y `docs/evidencia/pruebas-kotlin-conteo.txt`.

Pruebas de Jest

Jest es el corredor de pruebas del mundo JavaScript. Estas pruebas corren en Node, sin emulador y sin navegador. Las de pantalla usan Testing Library para React Native, que monta el componente en memoria y lo opera por el texto que ve el usuario: busca “Ingresar”, lo presiona y revisa qué aparece después. No se prueba contra la estructura interna del componente, sino contra lo que la persona vería.

El puente nativo no existe en Node, así que lo reemplacé por un doble en `tests/setup/nativeModules.ts`. Ese archivo registra un `BilleteraBridge` falso cuyos métodos son funciones espía, y cifra las respuestas con la misma llave AES y el mismo formato de sobre que usa Kotlin. Es decir: las pantallas de prueba reciben datos cifrados de verdad, no objetos sueltos. Cuando una prueba quiere verificar qué se le mandó al nativo, descifra el sobre que quedó guardado en la función espía.

```
npm test
```

```
> billetera@0.0.1 test
> jest

PASS tests/formato.test.ts
PASS tests/validaciones.test.ts
PASS tests/crypto.test.ts
PASS tests/agruparPorDia.test.ts
PASS tests/useTransferencia.test.ts
PASS tests/MovimientosBundle.test.tsx
PASS tests/LoginBundle.test.tsx
PASS tests/HomeBundle.test.tsx

Test Suites: 8 passed, 8 total
Tests:       43 passed, 43 total
Snapshots:   0 total
Time:        3.969 s
Ran all test suites.
```

8 archivos, 43 pruebas, 43 en verde. Las corrí dos veces, al empezar la verificación y en la pasada final, con el mismo resultado: 3,969 segundos la primera vez y 3,87 la segunda.

Archivo	Pruebas	Qué comprueba
crypto.test.ts	8	El canal cifrado del lado de JavaScript y el vector compartido con Kotlin
useTransferencia.test.ts	5	La máquina de pasos de la transferencia, del celular al resultado
LoginBundle.test.tsx	3	La pantalla de login: campos vacíos, credenciales malas y el botón que no se queda girando
HomeBundle.test.tsx	2	El saludo tras el aviso HOME_READY y el cierre de sesión

		delegado al contenedor
MovimientosBundle.test.tsx	2	La lista agrupada por día y el caso de la cuenta sin movimientos
agruparPorDia.test.ts	4	El agrupamiento por día, ordenado y sin modificar la entrada
formato.test.ts	6	Pesos colombianos, fechas y las etiquetas Hoy y Ayer
validaciones.test.ts	13	Celular, monto y el formateo del monto mientras se escribe

Qué cubre cada archivo

El canal cifrado: crypto.test.ts, 8 pruebas

- Cifrar y descifrar un payload con el IV que entrega el nativo, y comprobar que el sobre tiene exactamente las claves iv y data.
- Usar tal cual el IV que devolvió randomBytes del lado nativo, sin generarlo en JavaScript.
- Un tag alterado no descifra, y el mensaje de error no arrastra nada del payload.
- Un sobre que ni siquiera es JSON tampoco pasa.
- Contra el vector compartido: descifrar el sobre del archivo y, con la misma llave y el mismo IV, producir exactamente el mismo sobre.
- Los dos últimos casos corren con TextDecoder borrado del objeto global, porque Hermes no lo trae. Ahí se prueban tildes y un carácter de cuatro bytes, que es donde suele romperse una implementación hecha a mano.

```

src > shared > TS crypto.ts > ...
1 import {gcm} from '@noble/ciphers/aes.js';
2 import {utf8ToBytes} from '@noble/ciphers/utls.js';
3 import {base64} from '@scure/base';
4 import {NativeModules} from 'react-native';
5
6 let clave: Uint8Array | null = null;
7
8 export function setClaveCanal(b64: string): void {
9   | clave = base64.decode(b64);
10 }
11
12 export async function cifrar(obj: unknown): Promise<string> {
13   const iv = await NativeModules.BilleteraBridge.randomBytes(12);
14   return cifrarConIv(obj, base64.decode(iv));
15 }
16
17 // Solo la usan las pruebas del vector compartido. En la app el IV siempre sale
18 // del SecureRandom de Android, nunca de aquí.
19 export function cifrarConIv(obj: unknown, iv: Uint8Array): string {
20   const data = gcm(llaveActual(), iv).encrypt(utf8ToBytes(JSON.stringify(obj)));
21   return JSON.stringify({iv: base64.encode(iv), data: base64.encode(data)});
22 }
23
24 export function descifrar(envelope: string): unknown {
25   const llave = llaveActual();
26   try {
27     const sobre = JSON.parse(envelope);
28     const plano = gcm(llave, base64.decode(sobre.iv)).decrypt(
29       | base64.decode(sobre.data),
30     );

```

Figura 2. El lado de JavaScript del mismo canal. Con este código crypto.test.ts descifra el sobre que produjo Kotlin y reproduce el suyo byte a byte.

La transferencia: useTransferencia.test.ts, 5 pruebas

useTransferencia es el hook que lleva la pantalla de transferencia por sus pasos: destino, monto, confirmación y resultado. Un hook, en React, es la función que guarda el estado de una pantalla y expone las acciones que la mueven.

- El camino completo, del celular al resultado. La prueba descifra lo que se le pasó al puente y verifica que sea el trío celular, monto y descripción, y que el evento TRANSFER_SUCCESS salga con el identificador del movimiento, también cifrado.
- Un celular incompleto ni siquiera cruza el puente: el aviso lo da la validación local.
- Un celular sin cuenta deja el paso donde estaba y muestra “Ese celular no tiene cuenta”.
- Un monto en cero no abre la hoja de confirmación.
- Con saldo insuficiente la hoja se cierra, el paso vuelve al monto, no se emite TRANSFER_SUCCESS y al corregir el monto el error desaparece.

Las tres pantallas: 7 pruebas

- Login: con los campos vacíos avisa y no llama al puente. El código CREDENCIALES_INVALIDAS se traduce a “Celular o clave incorrectos”. Y si el contenedor no navega en cinco segundos, el botón deja de girar y aparece un aviso, en vez de quedarse colgado.
- Home: al montarse emite HOME_READY y muestra “Cargando”. Cuando llega LOAD_HOME cifrado, pinta “Hola, Alex” y “\$ 1.250.000”. El botón de cerrar sesión no borra nada por su cuenta: emite LOGOUT y deja que Android decida.
- Movimientos: separa los de hoy de los de ayer, formatea los montos con su signo y muestra el estado “Rechazada” junto a los dos “Exitosa”. Con la cuenta sin movimientos muestra el aviso y ninguna fila.

Formato, validaciones y agrupación: 23 pruebas

- formato.test.ts: pesos colombianos con punto de miles, el signo antes del símbolo, fechas con mes abreviado y hora de 24, y las etiquetas Hoy y Ayer resueltas contra un reloj congelado.
- validaciones.test.ts: el celular de diez dígitos que empieza por 3, el monto entero y positivo, y el formateo del monto mientras se escribe.
- agruparPorDia.test.ts: una sección por día, de la más reciente a la más vieja, y el arreglo de entrada intacto después de agrupar.

En la misma pasada quedaron `npx tsc --noEmit` y `npm run lint` en cero errores. Las salidas completas están en `docs/evidencia/pruebas-jest.txt`, `docs/evidencia/pasada-final-js.txt`, `docs/evidencia/typecheck.txt` y `docs/evidencia/lint.txt`.

Pruebas instrumentadas en el emulador

Estas pruebas se instalan en el emulador y corren allá adentro, dentro del proceso de la aplicación. Gradle compila dos APK: el de la aplicación y el de las pruebas. El corredor es AndroidJUnitRunner y las pantallas se manejan con ActivityScenario, que abre una Activity, la lleva por su ciclo de vida y la cierra al terminar.

Lo que aportan frente a las unitarias es justamente lo que no se puede simular en el computador.

- El Android Keystore de verdad. SecureStoreKeystoreTest cifra con una llave que el sistema guarda aparte y que nunca sale de ahí. En la JVM del computador esa llave no existe, así que la comprobación solo tiene sentido en el dispositivo.
- La carga real del bundle. LoginActivityCargaBundleTest y HomeActivityCargaBundleTest esperan a que el contexto de React se inicialice y a que el ReactRootView tenga hijos. Eso es Hermes leyendo el archivo de assets y ejecutándolo de verdad.
- La navegación entre pantallas. NavegacionDesdeHomeTest empuja los eventos OPEN_TRANSFER y OPEN_MOVEMENTS y comprueba que Android abra la Activity correcta.
- Las decisiones que ocurren dentro de onCreate. HomeSinSesionRedirigeTest lanza Home sin sesión y verifica que el usuario termine en Login. Esa rama cierra la pantalla antes de que alcance a mostrarse, y no hay forma de verla sin un Android corriendo.

```
cd android
gradlew connectedDebugAndroidTest
```

```
> Task :app:connectedDebugAndroidTest
Starting 13 tests on anden_demo(AVD) - 16

anden_demo(AVD) - 16 Tests 2/13 completed.
(0 skipped) (0 failed)
anden_demo(AVD) - 16 Tests 12/13 completed.
(0 skipped) (0 failed)
Finished 13 tests on anden_demo(AVD) - 16

BUILD SUCCESSFUL in 1m 54s
79 actionable tasks: 21 executed, 58 up-to-date
```

13 pruebas en emulator-5554, 0 fallos, 0 omitidas. Pasaron al primer intento y las repetí en la pasada final sin tocar nada: 24,8 segundos la primera corrida y 23,4 la segunda, contando solo la ejecución de las pruebas. Lo que tarda de más la compilación es armar los dos APK.

Clase	Pruebas	Qué comprueba
SecureStoreKeystoreTest	3	Guarda y recupera con la llave del Keystore, el valor crudo no es el texto plano, y un valor manipulado devuelve nulo y se borra
SesionPersistenciaTest	3	Otra instancia lee la sesión guardada, lo guardado no lleva la clave, y cerrar borra la entrada

SplashRedirigeALoginTest	1	Sin sesión, el splash termina en Login
LoginActivityCargaBundleTest	1	El bundle de Login monta, pinta y recibe la llave del canal en las props iniciales
HomeActivityCargaBundleTest	1	Con sesión válida, Home monta el bundle y acepta el aviso HOME_READY
HomeSinSesionRedirigeTest	1	Sin sesión, Home no se abre y el usuario termina en Login
NavegacionDesdeHomeTest	2	OPEN_TRANSFER abre Transferencia y OPEN_MOVEMENTS abre Movimientos
LogoutLimpiaSesionTest	1	Tras LOGOUT la sesión queda nula y aparece Login

Cómo se espera a React Native sin pruebas frágiles

El punto delicado de probar React Native desde Android es el tiempo: el bundle carga en un hilo aparte y el contenido aparece después. Una prueba que duerma tres segundos y asuma que ya está listo falla el día que la máquina esté ocupada. Puse todas las esperas en un solo objeto, `EsperaDeBundle`, con tres funciones.

- `contextoListo` se cuelga de `addReactInstanceEventListener` y espera el aviso `onReactContextInitialized`. Espera un evento, no un número de segundos.
- `contenidoPintado` consulta el `childCount` del `ReactRootView` hasta que deje de ser cero. Que el contexto exista no quiere decir que el componente ya haya pintado.
- `pantallaAbierta` registra un monitor de instrumentación antes de disparar la navegación, porque si la Activity alcanza a abrirse primero, la espera se agota sin haberla visto. La pantalla que aparezca se cierra ahí mismo, para no dejarla encima de la siguiente prueba.

En el informe XML de la segunda corrida las 13 pruebas quedan agrupadas bajo un solo elemento `testsuite`, que Gradle nombra con la primera clase que ejecutó. Los atributos que importan son `tests="13"`, `failures="0"`, `errors="0"` y `skipped="0"`. El informe en HTML queda en `android/app/build/reports/androidTests/connected/` y las salidas de consola en `docs/evidencia/instrumentadas-intento1.txt`, `instrumentadas-informe.txt` y `pasada-final-gradle.txt`.

El recorrido completo en capturas

Estas once capturas son el recorrido completo sobre el emulador, en el orden en que ocurre. Salieron con `adb shell screencap` sobre el APK de depuración. De cada pantalla quedó además un archivo de texto con lo que se leyó en ella, en `docs/evidencia/capturas-1.txt` y siguientes.



Figura 3. Splash nativo, escrito en Kotlin. Mientras se ve la marca, Android revisa el dispositivo y decide si va a Login o directo a Home.

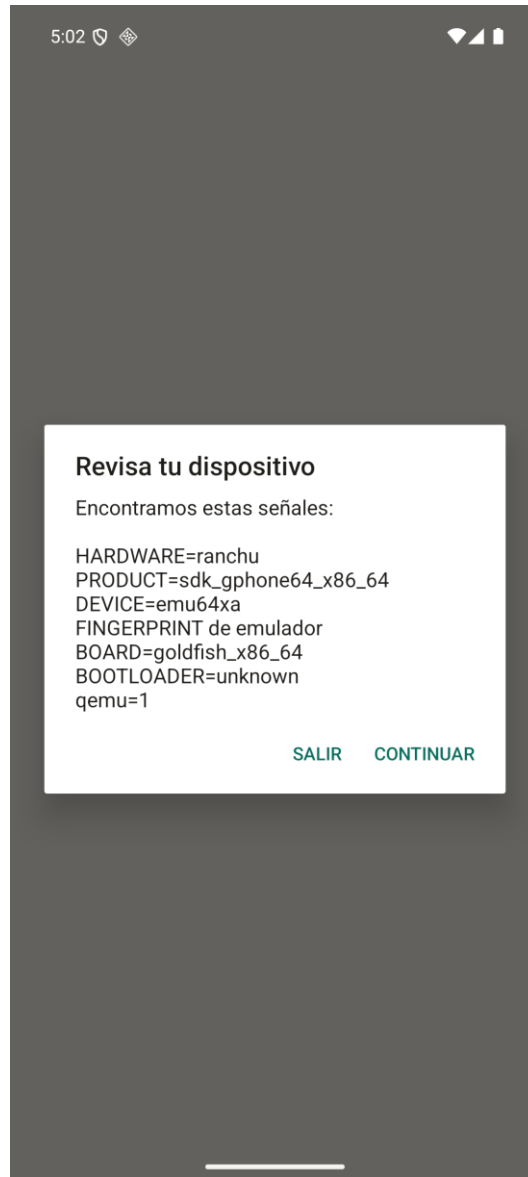


Figura 4. La revisión de integridad encontró que esto es un emulador y lo dice, con las señales que lo delatan: *HARDWARE=ranchu, PRODUCT=sdk_gphone64_x86_64, qemu=1*. En depuración el aviso deja continuar.

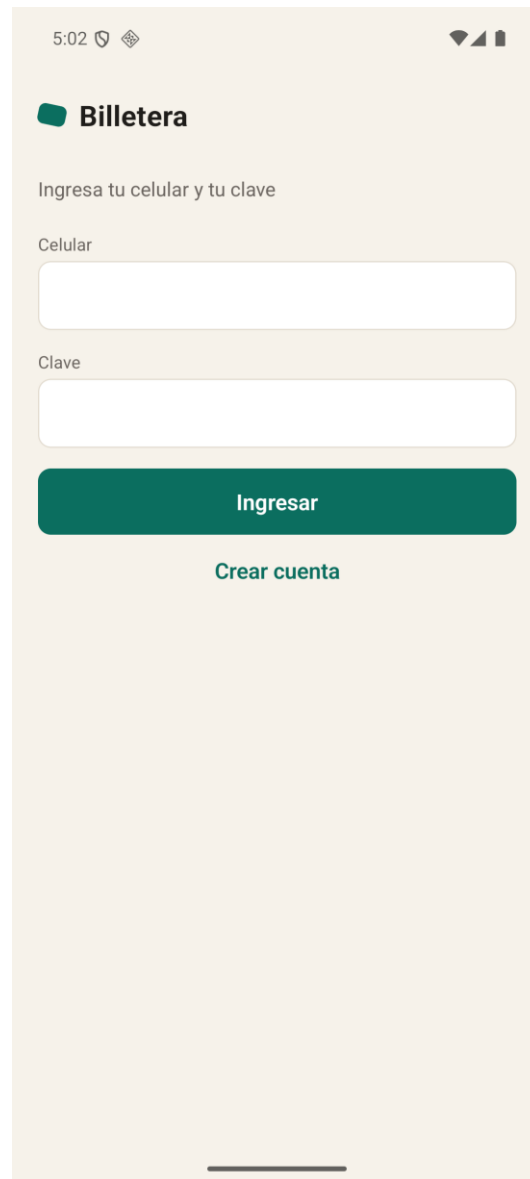


Figura 5. LoginActivity con login.android.bundle cargado desde assets. Es la primera prueba mínima del enunciado: el bundle carga y pinta. Es además el único bundle público, el que no exige sesión.

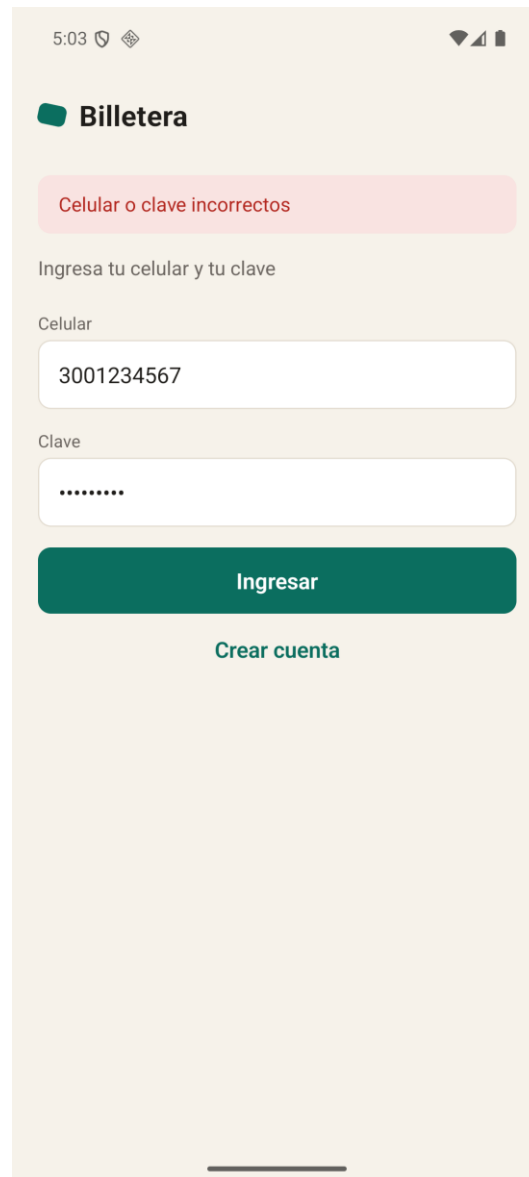


Figura 6. Login fallido. El lado nativo responde `CREDENCIALES_INVALIDAS` y el bundle lo traduce a "Celular o clave incorrectos". El mismo texto sale si el celular no existe, para no revelar qué números tienen cuenta.

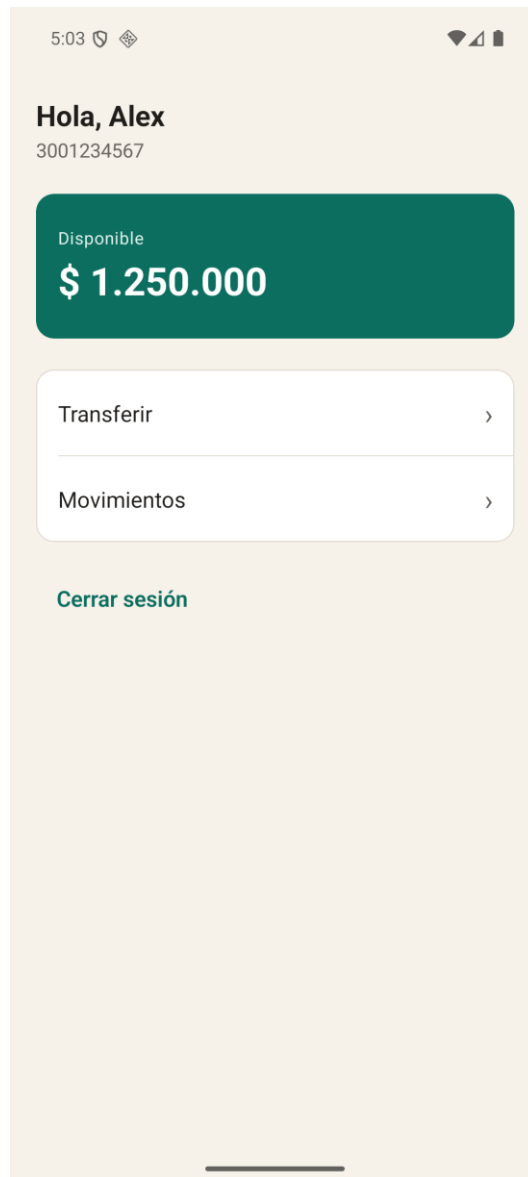


Figura 7. Login exitoso, sesión creada y bundle de Home montado. El nombre, el celular y el saldo no vienen en las props iniciales: llegan cifrados en el evento LOAD_HOME, después de que el bundle avisa con HOME_READY que ya está escuchando.

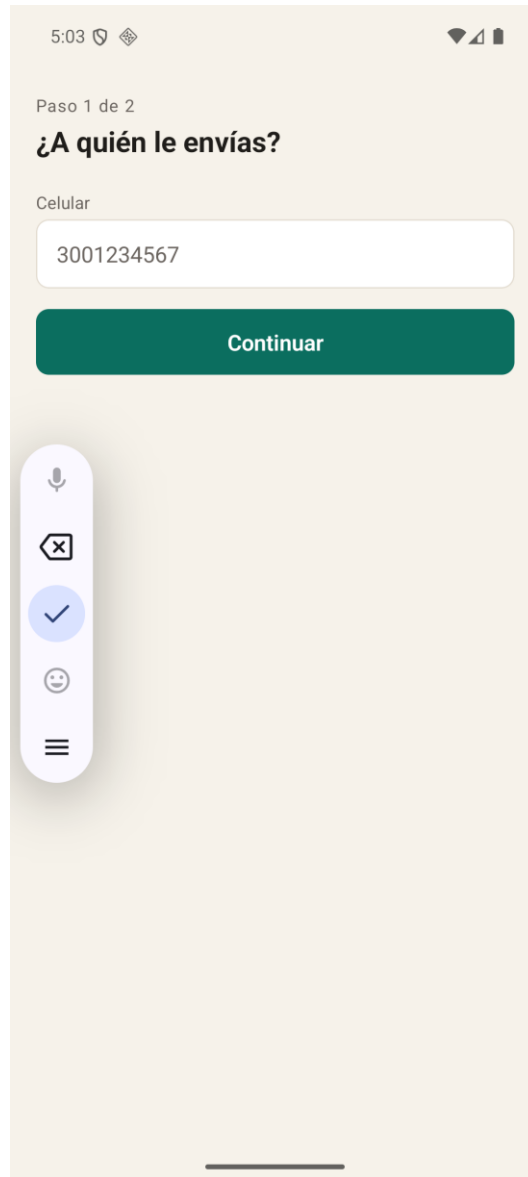


Figura 8. El evento OPEN_TRANSFER desde Home abrió TransferenciaActivity con su propio bundle. Paso 1 de 2: el celular del destinatario.

5:04

Paso 2 de 2

¿Cuánto le envías?

Laura P.
3109876543 [Cambiar](#)

Monto en pesos

200.000

Descripción (opcional)

Pago del almuerzo

[Continuar](#)

Revisa antes de enviar

Para	Laura P.
Celular	3109876543
Descripción	Pago del almuerzo

Monto	\$ 200.000
-------	-------------------

[Confirmar](#)

[Cancelar](#)

Figura 9. El destino se resolvió contra el backend simulado y vuelve enmascarado como "Laura P.", nunca con el nombre completo. La hoja de confirmación muestra lo que se va a enviar antes de enviarlo.

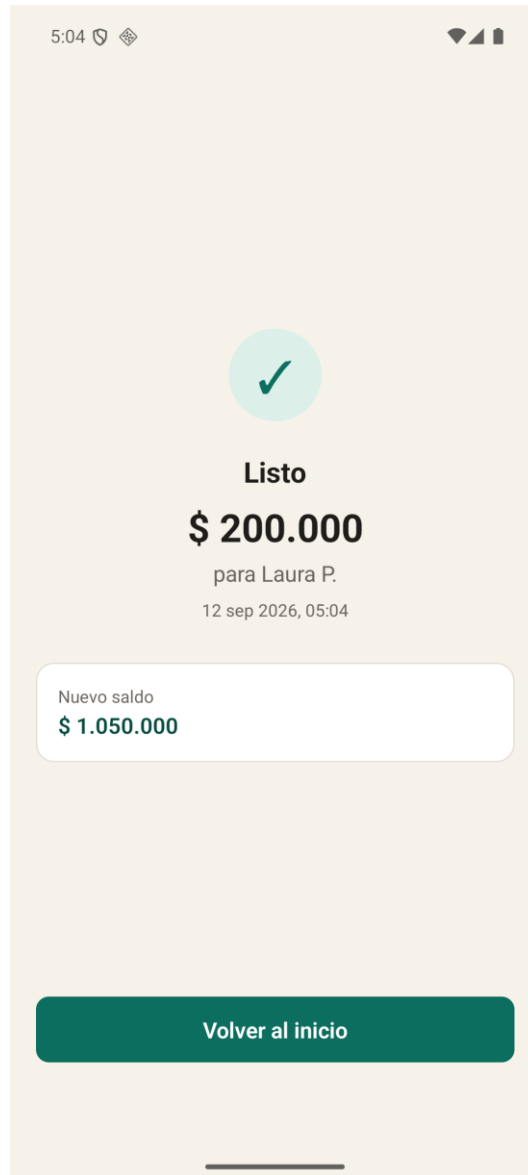


Figura 10. Transferencia exitosa de \$150.000. El nuevo saldo, \$1.100.000, viene en la respuesta del lado nativo. JavaScript no lo calcula, solo lo muestra.

5:08

Paso 2 de 2

¿Cuánto le envías?

Laura P.
3109876543 [Cambiar](#)

Monto en pesos

99.000.000

Descripción (opcional)

Almuerzo

No tienes saldo suficiente

Continuar

Figura 11. Transferencia rechazada por saldo: \$9.000.000 sobre un saldo de \$1.100.000. El nativo responde SALDO_INSUFICIENTE, la hoja de confirmación se cierra y el monto queda editable para corregirlo.

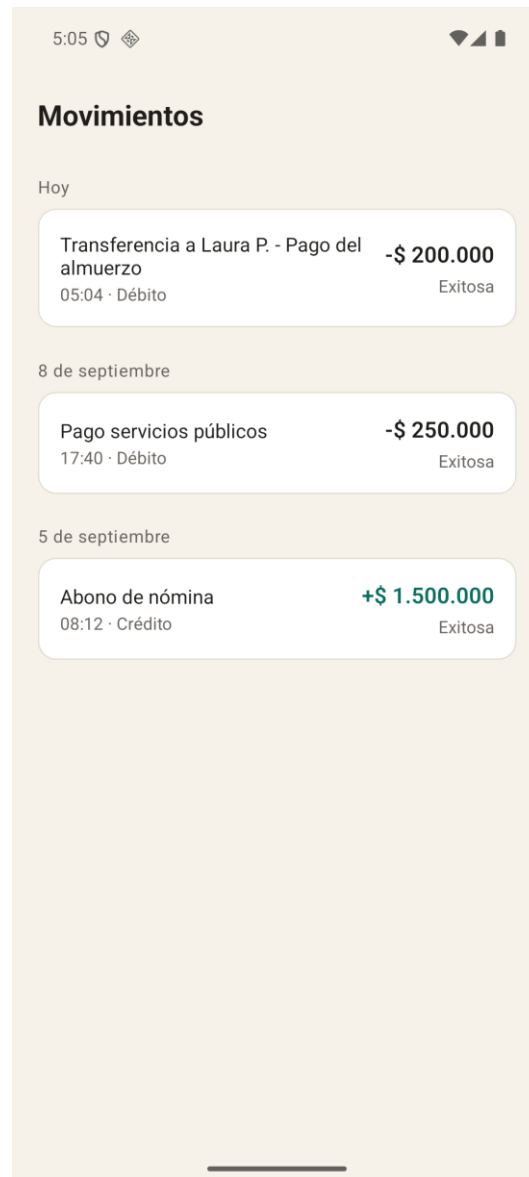


Figura 12. Movimientos del usuario autenticado, agrupados por día y con el más reciente primero. Arriba está el intento rechazado, que también se registra como movimiento aunque no mueva plata.

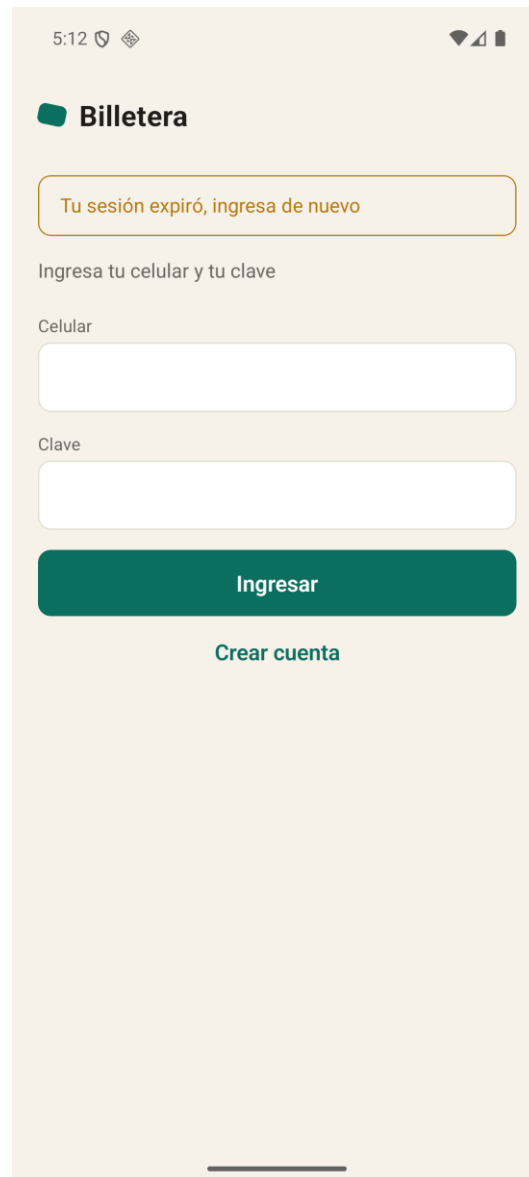


Figura 13. Sesión expirada. Con el tiempo de vida de depuración, que son dos minutos, dejé Home abierto hasta que venció. Android emite `SESSION_EXPIRED`, el bundle muestra el aviso y dos segundos después la aplicación está en Login con el texto “Tu sesión expiró, ingresa de nuevo”.

La expiración la seguí con una ráfaga de capturas, una cada pocos segundos, anotando qué Activity tenía el foco. En el fotograma 30 el foco ya era `LoginActivity`. Eso quedó en `docs/evidencia/capturas-5-expiracion.txt` y `capturas-6-login-tras-expirar.txt`.

La compilación de publicación

El APK de publicación se compila con `gradlew assembleRelease`. Esa variante hace tres cosas que la de depuración no hace: pasa el código por R8, lo firma con una llave propia y sube el tiempo de vida de la sesión de 2 a 10 minutos.

R8 renombró y borró

R8 es el ofuscador y reductor de código de Android. Renombra clases y métodos a nombres cortos y borra lo que nadie usa. En el módulo está activado con `minifyEnabled true` y `shrinkResources true`, con las reglas de `proguard-android.txt` más las propias del proyecto en `proguard-rules.pro`.

El `mapping.txt` que deja la compilación tiene 113.995 líneas. Es el diccionario que traduce los nombres nuevos a los originales; sin él, un reporte de error de un APK ofuscado es ilegible. De las 70 clases propias del paquete `com.asuarez.billetera`, 62 quedaron con un nombre distinto. Este es un extracto real del archivo.

```
com.asuarez.billetera.data.BancoMockApi -> P.b:
com.asuarez.billetera.data.MockDb -> P.f:
com.asuarez.billetera.data.Movimiento -> P.g:
com.asuarez.billetera.bundles.BundleHost -> O.j:
com.asuarez.billetera.bundles.RnBundle -> O.o:
com.asuarez.billetera.bridge.JsEmitter -> N.d:
```

Las ocho clases que conservan el nombre no son un descuido. Siete las nombra el `AndroidManifest.xml` como texto: la clase `BilleteraApp` y las seis `Activities`, `SplashActivity`, `LoginActivity`, `HomeActivity`, `TransferenciaActivity`, `MovimientosActivity` y `ErrorActivity`. La octava es `BilleteraBridgeModule`, que React Native busca por nombre desde JavaScript. Si R8 renombrara cualquiera de esas, la aplicación no arrancaría. El resumen está en `docs/evidencia/r8-mapping.txt`.

El APK va firmado

Un APK sin firmar no se instala. La firma dice quién lo publicó y garantiza que nadie lo modificó después de compilarlo.

El primer `assembleRelease` sí generó APK, pero cayendo a la llave de depuración, con este aviso en la consola: “Sin `keystore.properties`: release firmado con la llave de depuración, solo para evaluación”. Eso quedó en `docs/evidencia/build-release-intento1.txt`, y lo dejo a la vista a propósito, porque muestra que la compilación avisa en vez de fingir. Después generé un `keystore` de evaluación con `keytool`, RSA de 2.048 bits y validez de 10.000 días, y volví a compilar. La verificación con `apksigner` quedó así.

```
Verifies
Verified using v1 scheme (JAR signing): false
Verified using v2 scheme (APK Signature Scheme v2): true
Number of signers: 1
```

```
Signer #1 certificate DN: CN=Billetera
Signer #1 key algorithm: RSA
Signer #1 key size (bits): 2048
```

Un solo firmante, CN=Billetera, con el esquema v2, que firma el archivo completo y no entrada por entrada. El .jks y el keystore.properties con las contraseñas no están en el repositorio: docs/evidencia/gitignore-verificacion.txt muestra git check-ignore confirmando que los tres archivos sensibles están ignorados.

Los permisos de cada variante

El APK de publicación declara un solo permiso, y no es de los que el usuario tiene que aprobar: com.asuarez.billetera.DYNAMIC_RECEIVER_NOT_EXPORTED_PERMISSION, que genera el propio Android para los receptores dinámicos. El de depuración además lleva INTERNET y SYSTEM_ALERT_WINDOW, que vienen del menú de desarrollo de React Native y del servidor Metro. La aplicación no pide red, ni contactos, ni almacenamiento. La comprobación está en docs/evidencia/permisos-apk.txt.

El recorrido completo repetido sobre el APK ofuscado

Compilar con R8 puede romper cosas que funcionaban: una clase que se buscaba por nombre y que ya no se llama así, un campo que se borró por parecer inútil. Por eso repetí el recorrido completo sobre el APK de publicación instalado en el emulador, con los datos en limpio. Como FLAG_SECURE impide la captura, leí cada pantalla con uiautomator y anoté qué Activity tenía el foco y qué textos había.

Paso	Pantalla	Lo que confirma
1	SplashActivity	El aviso de dispositivo con las señales del emulador
2	LoginActivity	El bundle de Login monta bajo el código ofuscado
3	LoginActivity	Clave equivocada: "Celular o clave incorrectos"
4	HomeActivity	"Hola, Alex" y el saldo de \$ 1.250.000
5	TransferenciaActivity	Paso 1 de 2, el celular del destino
6	TransferenciaActivity	Paso 2 de 2, con el destino resuelto como "Laura P."
7	TransferenciaActivity	Hoja de confirmación por \$ 150.000
8	TransferenciaActivity	Resultado con el nuevo saldo de \$ 1.100.000
9	HomeActivity	Home ya con el saldo actualizado
10	TransferenciaActivity	"No tienes saldo suficiente" al intentar 9.000.000
11	HomeActivity	Vuelta a Home sin perder la sesión

12	MovimientosActivity	La lista con el movimiento rechazado de primero
13	LoginActivity	Logout: la aplicación vuelve a Login

Así se ve uno de esos pasos en el archivo. La línea de la pantalla va partida aquí para que quepa en la página.

```
--- 4. HOME ---
actividad: com.asuarez.billetera/.bundles.HomeActivity
pantalla : Hola, Alex | 3001234567 | Disponible |
          $ 1.250.000 | Transferir | Movimientos |
          Cerrar sesión
```

El registro del sistema durante ese recorrido no tiene un solo cierre inesperado del paquete. Se ven los cuatro bundles arrancando y el registro de la transferencia. Estas líneas van sin la marca de tiempo ni el número de proceso, para que quepan.

```
I ReactNativeJS: Running "LoginBundle"
I ReactNativeJS: Running "HomeBundle"
I ReactNativeJS: Running "TransferenciaBundle"
I ReactNativeJS: Running "MovimientosBundle"
I Billetera: Transferencia registrada: m00004
```

Las salidas están en docs/evidencia/smoke-release-1-login.txt hasta smoke-release-6-movimientos-logout.txt, más logcat-release.txt e instalacion-release.txt.